

FIȘĂ DE LUCRU - Liste în Python: metode și operații

Fișe de lucru - Informatică, clasa a IX-a | Python

Informatică • Clasa a IX-a • Python

Date generale

Disciplina: Informatică | **Clasa:** a IX-a | **Durată:** 50 minute | **Tip:** fișă de lucru – teorie, exemplu, problemă

Tipul 1. Metode de bază ale listei: `append()`, `insert()`, `pop()`

Partea teoretică. Clasa `list` din Python pune la dispoziție metode pentru adăugarea și eliminarea elementelor.

- `append(x)` — adaugă elementul `x` la **sfârșitul** listei.
- `insert(i, x)` — inserează elementul `x` pe **poziția** `i`, deplasând restul spre dreapta.
- `pop(i)` — **elimină și returnează** elementul de pe poziția `i`. Dacă poziția nu este precizată, elimină ultimul element.

⚠ Atenție!

- `append()` adaugă întotdeauna la **sfârșitul** listei.
- `insert()` permite alegerea oricărei poziții.
- `pop()` elimină elementul și **returnează** valoarea eliminată.

✓ Exemplu rezolvat – Gestionăți lista participanților la un eveniment.

Adăugăm participanți cu `append()`, inserăm un invitat pe prima poziție cu `insert(0, ...)` și eliminăm ultimul participant cu `pop()`.

```
participanti = []

participanti.append("Ana")
participanti.append("Mihai")
participanti.append("Elena")

print("Lista inițială:", participanti)

participanti.insert(0, "Prof. Ionescu") # inserare pe poziția 0

print("După insert:", participanti)

eliminat = participanti.pop()          # elimină ultimul element

print("Element eliminat:", eliminat)
print("Lista finală:", participanti)
```

➡ Problemă propusă – O cofetărie gestionează comenzile zilei.

- creați o listă goală;
- adăugați trei comenzi folosind `append()`;
- inserați o comandă urgentă pe prima poziție;
- eliminați ultima comandă folosind `pop()`;
- afișați lista finală.

Indicație: utilizați aceeași structură ca în exemplu.

Tipul 2. Căutare în listă: `index()` și `count()`

Partea teoretică. Metodele `index()` și `count()` permit căutarea valorilor în liste.

- `index(x)` — returnează **poziția** primei apariții a elementului `x`.
- `count(x)` — returnează **numărul de apariții** ale elementului `x` în listă.

⚠ Atenție!

Dacă valoarea căutată nu există în listă, `lista.index(x)` generează eroarea `ValueError`.
Verificați existența elementului cu `x in lista` înainte de a apela `index()`.

✓ Exemplu rezolvat – Analizați stocul unui magazin.

Determinăm poziția unui produs și numărul de apariții al altuia.

```
stoc = ["mere", "pere", "mere", "banane", "mere", "pere"]

pozitie = stoc.index("banane")
print("Bananele se află pe poziția:", pozitie)

numar = stoc.count("mere")
print("Mere apar de", numar, "ori.")
```

➡ Problemă propusă – Verificați prezența elevilor.

Se citește o listă cu numele elevilor prezenți la un test.

- afișați pe ce poziție se află un elev dat (citit de la tastatură);
- afișați de câte ori apare un alt nume în listă.

Indicație: folosiți `index()` și `count()` cu variabile de tip `str`.

Tipul 3. Sortarea și inversarea unei liste

Partea teoretică. Python oferă mai multe modalități de ordonare a elementelor dintr-o listă:

- `sort()` — ordonează lista **în loc** (modifică lista originală), implicit crescător.
- `sorted(lista)` — returnează o **listă nouă** ordonată, fără să modifice lista originală.
- `reverse()` — **inversează** ordinea elementelor din lista originală.

`sort()` și `sorted()` acceptă parametrul `reverse=True` pentru ordine descrescătoare. Metoda `reverse()` nu primește parametri; ea doar inversează ordinea elementelor din listă.

⚠ Atenție!

- `sort()` **modifică** lista originală — nu poate fi anulat fără o copie prealabilă.
- `sorted()` **nu modifică** lista originală — rezultatul trebuie salvat într-o variabilă.
- Folosiți `sorted()` când doriți să păstrați ordinea inițială intactă.

✓ Exemplu rezolvat – Ordonăți punctajele participanților la un concurs.

Afișăm clasamentul descrescător fără a modifica lista originală, apoi sortăm în loc și inversăm.

```
punctaje = [85, 92, 71, 88, 95, 60]

clasament = sorted(punctaje, reverse=True)
print("Clasament:", clasament)
print("Lista originală:", punctaje)      # rămâne neschimbată

punctaje.sort()
print("Sortată crescător:", punctaje)

punctaje.reverse()
print("Ordine inversă:", punctaje)
```

➔ Problemă propusă – Gestionăți notele dintr-un catalog.

Se citește o listă cu notele elevilor.

- afișați notele sortate crescător **fără a modifica lista inițială**;
- afișați nota minimă și nota maximă;
- afișați lista în ordinea inversă față de cea în care au fost introduse.

Indicație: folosiți `sorted()`, `min()`, `max()` și `reverse()`.

Tipul 4. Copierea unei liste

Partea teoretică. Atribuirea directă `b = a` **nu creează o copie** a listei — ambele variabile vor referi aceeași zonă de memorie. Orice modificare în `b` o va afecta și pe `a`. Pentru o copie independentă se utilizează:

- `b = a.copy()` — metodă explicită de copiere.
- `b = a[:]` — slicing care returnează o copie a întregii liste.

⚠ Atenție!

După `b = a`, ambele variabile se referă la **aceeași** listă. Modificarea lui `b[0]` va modifica și `a[0]`. Folosiți `copy()` sau `[:]` pentru liste independente.

✓ Exemplu rezolvat – Comparați două versiuni ale unui stoc.

Pornim de la un stoc, facem o copie reală, modificăm copia și verificăm că originalul rămâne intact.

```
stoc = [10, 20, 30, 40]

copie = stoc.copy()

copie[0] = 99

print(stoc)      # [10, 20, 30, 40]
print(copie)     # [99, 20, 30, 40]
```

Problemă propusă – Simulați o reducere de prețuri.

- creați o copie independentă a listei de prețuri;
- reduceți fiecare preț din copie cu 10%;
- afișați lista inițială și lista modificată.

Indicație: `b = a.copy()`, apoi `for` cu `b[i] *= 0.9`.

Autoevaluare

- ☐ Pot utiliza metodele `append()`, `insert()` și `pop()` pentru adăugarea și eliminarea elementelor.
- ☐ Pot utiliza `index()` și `count()` pentru căutarea valorilor într-o listă.
- ☐ Cunosc diferența dintre `sort()` (modifică lista) și `sorted()` (returnează o copie).
- ☐ Înțeleg diferența dintre copiere independentă (`copy()`, `[:]`) și atribuire directă (`b = a`).

Temă opțională

Alegeți una dintre problemele propuse și scrieți programul complet în Python. Testați programul pentru:

- un caz obișnuit;
- o listă goală sau cu un singur element;
- o listă cu valori repetate.